STUDY GUIDE · PRACTICE Q&A

Databricks Certified Data Engineer Associate

78 **unique questions**
with explanations

Duplicates removed from a 1,030-question bank and organized by the 7 official exam sections.

+ 28 hands-on SQL & PySpark code questions

EXAM WEIGHT BY SECTION



COMPILED & EDITED BY

Stanly Fernández

Business Intelligence & Data Analytics

Unofficial study material. Not affiliated with or endorsed by Databricks, Inc.

BEFORE YOU START

Credits & how to use this guide

Credits

Compiled & edited by Stanly Fernández. Deduplication, regrouping by exam section, wording clean-up, editor's notes and layout.

Source material: the practice questions and explanations come from cert-pass.com, *"Databricks Data Engineer Associate: Complete Q&A with Explanations"* (1,030 questions). All credit for the original questions and explanations goes to that source.

Exam outline and weights: Databricks Certified Data Engineer Associate page on databricks.com, checked on October 18, 2026.

What was changed

- Grouped the 1,030 questions by identical correct answer and explanation. Most only changed the industry, data source, cloud or a "Scenario variant" tag, so each concept was kept once.
- Removed the generic scenario openers and kept the technical question and its four options.
- Re-ordered everything into the 7 official exam sections, with short topic groups inside each one.
- Added a few editor's notes, and **Part B: 28 original hands-on SQL, PySpark and CLI questions** written by the editor (the source bank has no code-reading items).

How to study with it

1. Read the question and answer before you look at the options marked as correct.
2. Check the explanation, then say why each wrong option fails. That is how the exam distractors work.
3. Spend time in proportion to the weights on the next page, and try each feature in a real workspace.

Important This is a practice set of **78 unique questions** from the source plus **28 editor-written code questions**. It does not cover every exam objective, and these are not real exam questions. Use it with the official exam guide and the Databricks documentation. Source answers were not re-verified one by one, except where an editor's note says otherwise. Part B answers follow current Databricks documentation, so test them in a workspace.

TABLE OF CONTENTS

Exam sections, topics and questions

	SECTION AND TOPICS	EXAM WEIGHT	APPROX. EXAM Q'S	QUESTIONS IN GUIDE	PAGE
1	**Databricks Intelligence Platform** Lakehouse & Delta Lake · Compute · Unity Catalog basics	6%	≈ 3	8 Q1-Q8	5
2	**Data Ingestion and Loading** Auto Loader · COPY INTO & ingestion patterns · Connectors & APIs · Landing zone & governed targets · Data-driven triggers	21%	≈ 9	12 Q9-Q20	9
3	**Data Transformation and Modeling** Medallion architecture & data quality · Joins & unions · Transforming data · Spark performance basics · Writing & serving data	22%	≈ 10	16 Q21-Q36	15
4	**Working with Lakeflow Jobs** Task design & orchestration · Retries, triggers & monitoring · Control flow & parameters	16%	≈ 7	10 Q37-Q46	23
5	**Implementing CI/CD** Git & code quality · Bundles & deployment · Identity & secrets	10%	≈ 4-5	10 Q47-Q56	28
6	**Troubleshooting, Monitoring, and Optimization** Spark UI & performance issues · Cluster & dependency issues · Monitoring jobs & pipelines · Delta table optimization	10%	≈ 4-5	12 Q57-Q68	33
7	**Governance and Security** Managed vs external tables · Privileges & identities · Fine-grained access control · Governed data sharing	15%	≈ 7	10 Q69-Q78	39
B	**Hands-on code practice** SQL · PySpark · Auto Loader · MERGE · Lakeflow · CLI · Unity Catalog, tagged by exam section	all	code	28 Q79-Q106	44

The real exam has **45 scored questions in 90 minutes** (plus unscored items), a $200 fee and a 2-year validity, according to databricks.com. "≈ exam questions" is just 45 × the section weight. The guide has more questions in some sections than the exam will, so weigh your study time by the % column.

QUICK REFERENCE

Names to know

Databricks renamed several products. Older courses and blog posts still use the first column, while the exam and this guide use the second.

OLDER NAME (STILL IN MANY COURSES)	CURRENT NAME
Delta Live Tables (DLT)	**Lakeflow Spark Declarative Pipelines**
Workflows / Jobs	**Lakeflow Jobs**
Databricks Asset Bundles (DABs)	**Declarative Automation Bundles**

The names above were checked against Databricks documentation and recent exam-prep references. When a product name in a question looks unfamiliar, map it to its old name before you decide it is a different feature.

SECTION 01 · 6% OF THE EXAM

Databricks Intelligence Platform

Q1

LAKEHOUSE & DELTA LAKE

A team needs a reliable lakehouse foundation for IoT sensor drops that supports rollback after bad writes, consistent access for SQL analysts, and minimal custom code. Which approach best fits Databricks Data Intelligence Platform principles?

A Use temporary views for all curated data because views automatically provide physical rollback and audit history.

B Keep only Parquet files in object storage and grant users direct cloud IAM access.

C Store curated data as unmanaged CSV files in DBFS and rely on folder naming for versions.

D Use Delta Lake tables governed by Unity Catalog so ACID transactions, time travel, lineage, and shared access controls support BI and AI workloads. ✓

CORRECT: D Why
Delta Lake with Unity Catalog provides transactional reliability, governed discovery, lineage, and consistent access. The strongest distractor, raw Parquet or CSV in storage, misses transaction logs, platform governance, and simple rollback.

Q2

LAKEHOUSE & DELTA LAKE

A bad load introduced duplicate records into `silver_sessions`. The team needs to recover a previous valid state quickly. What platform feature is most relevant?

A Grant SELECT to fewer users because permissions automatically remove corrupted rows.

B Change the cluster runtime version and rerun the dashboard query.

C Delete all files in the table path and reload from a local CSV backup.

D Use Delta time travel or restore capabilities after identifying the bad version. ✓

CORRECT: D Why
Delta Lake transaction history enables time travel and restore workflows. Permissions or runtime changes do not revert data contents.

Databricks Intelligence Platform

Q3

LAKEHOUSE & DELTA LAKE

The data engineering team is creating a governed source of truth for orders data. Which storage and registration approach is most aligned to the lakehouse?

A Store only JSON files in the workspace Files area.

B Store curated data in Delta tables registered in Unity Catalog. ✓

C Store data in notebook-scoped temporary views and export query results to CSV for consumers.

D Place all data in the DBFS root and manage sharing with cluster attach permissions.

CORRECT: B Why
Delta tables in Unity Catalog are governed, discoverable, and reliable. DBFS root or workspace files are not the right governed source-of-truth pattern.

Q4

COMPUTE

Analysts query curated events tables during the day while scheduled transformations run overnight. The platform team wants isolation and cost-aware compute. What should they choose?

A Use a SQL warehouse for analyst SQL workloads and job compute for scheduled ETL tasks. ✓

B Use one all-purpose interactive cluster for every analyst query and production ETL job.

C Use model serving compute because it is optimized for SQL dashboard concurrency.

D Use a single-node cluster for dashboards because it has the lowest possible startup time.

CORRECT: A Why
SQL warehouses are designed for SQL analytics concurrency, while job compute is suitable for scheduled workloads. A single all-purpose cluster mixes use cases and weakens isolation and cost control.

Databricks Intelligence Platform

Q5

COMPUTE

A BI team needs fast startup and concurrency for ad hoc SQL on `gold_sales_mart`. Which compute service is the best fit?

A	Use a job cluster because job clusters are designed for long-running interactive BI sessions.
B	Use Lakeflow Jobs as the query engine because Jobs replace SQL warehouses for ad hoc analytics.
C	Use Databricks SQL warehouse/serverless SQL when the workload is interactive SQL analytics with many concurrent users. ✓
D	Use an all-purpose cluster with a single worker to guarantee the highest concurrency.

CORRECT: C Why
SQL warehouses are optimized for SQL analytics and concurrent BI users. Job clusters are best for scheduled job runs, not always-on interactive analytics.

Q6

COMPUTE

A new audit logs workload includes interactive exploration, scheduled ETL, and BI consumption. Which decision rule best reflects Databricks compute best practice?

A	Select compute based on workload characteristics such as interactivity, concurrency, scheduling, startup needs, and cost model. ✓
B	Always use single-node clusters for production because they are simpler to govern.
C	Always use the largest all-purpose cluster to avoid tuning decisions.
D	Always use serverless SQL for streaming ingestion because it replaces streaming compute.

CORRECT: A Why
Compute choice depends on workload type and cost/performance needs. The top wrong answer overprovisions and ignores workload isolation.

Databricks Intelligence Platform

Q7

UNITY CATALOG BASICS

A company is standardizing access to `gold_ops_dashboard` and wants discoverability, ownership, and governed permissions across teams. Which organization pattern is most appropriate?

A	Create a cluster per table and rely on cluster permissions to define table ownership.
B	Create one workspace folder per business unit and store production tables as notebook outputs.
C	Use DBFS root paths as the main security boundary for governed datasets.
D	**Create catalogs and schemas in Unity Catalog and organize data by environment, domain, and sensitivity.** ✓

CORRECT: D Why
Unity Catalog provides the catalog/schema/table hierarchy for governance and discovery. Workspace folders or cluster permissions do not govern data objects consistently.

Q8

UNITY CATALOG BASICS

A dashboard based on `gold_ops_dashboard` suddenly changed after a production deployment. The engineer must identify upstream dependencies and recent modifications. What should be used first?

A	Search DBFS file names because table lineage is encoded in file names.
B	Inspect only the notebook revision history because it always shows every data write.
C	**Use Unity Catalog lineage and table history to trace upstream tables and recent writes.** ✓
D	Ask every user to manually report which query they ran.

CORRECT: C Why
Unity Catalog lineage and Delta history help trace dependencies and modifications. Notebook history alone may miss writes from jobs, SQL, or pipelines.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q9

AUTO LOADER

New JSON files for security telemetry arrive continuously in GCS. The team needs incremental ingestion with schema tracking and minimal custom code. What should they implement?

A Manually upload files through the workspace UI for every batch.

B Use Auto Loader with a checkpoint and schema location to incrementally ingest new files into a Unity Catalog Delta table. ✓

C Use COPY INTO without tracking loaded files because it always rereads all historical files.

D Run SELECT * from the cloud folder each hour and overwrite the target table.

CORRECT: B Why
Auto Loader is designed for scalable incremental file ingestion with checkpointing and schema management. Overwriting from a folder risks duplicates, missed files, and poor operational reliability.

Q10

AUTO LOADER

An Auto Loader pipeline for IoT sensor drops receives a new optional field. The engineer wants controlled schema evolution. What should be configured?

A Delete the checkpoint whenever a new column appears so Auto Loader restarts from scratch.

B Provide a schema location and configure schema evolution behavior so changes are tracked instead of silently corrupting the target. ✓

C Disable schema inference and cast every column to string permanently.

D Run VACUUM before each ingest to force schema changes to appear.

CORRECT: B Why
Auto Loader uses a schema location to track schema and can evolve columns according to configuration. Deleting checkpoints is unsafe and can duplicate or skip data.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q11

AUTO LOADER

A bucket receiving ERP snapshots now contains millions of files, and listing is becoming expensive. Which Auto Loader discovery mode should be preferred when supported?

A Use a SQL warehouse to send object storage events directly to tables.

B Use directory listing forever because it is always lower latency for huge directories.

C Use DBFS root copies because file notification only works for DBFS.

D Use file notification mode when the storage and cloud setup support it and low-latency scalable discovery is required. ✓

CORRECT: D Why
File notification mode avoids repeated large listings and improves scalable discovery. Directory listing can become costly on very large directories.

Q12

AUTO LOADER

An ingestion job for audit logs failed midway and must resume without duplicating already processed data. Which concept is central?

A Rename every file after processing because Delta always requires source file mutation.

B Use ZORDER on the source folder to mark consumed files.

C Use checkpointing to track progress and ensure the stream or incremental load resumes from the correct point. ✓

D Use a dashboard refresh schedule as the ingestion state store.

CORRECT: C Why
Checkpoints and ingestion metadata track progress for resumable processing. File renaming is not the primary Databricks state-management pattern.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q13

AUTO LOADER

The ERP snapshots JSON feed occasionally sends extra nested fields and malformed records. The engineer wants to avoid data loss while keeping the pipeline running. What is the best approach?

A Run VACUUM on the source files to isolate malformed records.

B Set mergeSchema to false and ignore all source changes forever.

C Use rescued data or controlled parsing options to capture unexpected fields or malformed records for later review. ✓

D Drop all records with any unexpected column before writing bronze.

CORRECT: C Why
Rescued data and parsing controls preserve problematic inputs for later analysis. Blindly dropping records creates silent data loss.

Q14

COPY INTO & INGESTION PATTERNS

A partner places daily CSV extracts for audit logs in S3. Files are moderate in volume, and the engineer wants a simple SQL-based incremental load into `bronze_clicks`. Which command is most appropriate?

A Use INSERT OVERWRITE from all files on every run to avoid maintaining state.

B Use VACUUM to load only new files into the table.

C Use OPTIMIZE to discover and append new cloud files.

D Use COPY INTO for idempotent incremental loading of files from cloud object storage into a Delta table. ✓

CORRECT: D Why
COPY INTO tracks loaded files for straightforward SQL-based incremental file ingestion. OPTIMIZE and VACUUM manage Delta layout/retention, not ingestion.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q15

COPY INTO & INGESTION PATTERNS

A team compares ingestion patterns for retail POS feeds: one feed has frequent high-volume file arrivals, another has a small daily extract. Which comparison is most accurate?

A Choose Auto Loader for high-volume incremental file arrival and COPY INTO for simpler periodic SQL file loads. ✓

B Choose manual uploads when governance is required because managed ingestion bypasses Unity Catalog.

C Choose COPY INTO for all streaming sources and Auto Loader only for one-time historical backfills.

D Choose SQL dashboards because dashboards automatically ingest source files.

CORRECT: A Why
Auto Loader suits scalable incremental file discovery, while COPY INTO is simple for SQL-based incremental file loads. Manual uploads are not a governed scalable pattern.

Q16

CONNECTORS & APIS

The team must ingest Workday data into Unity Catalog governed tables and wants minimal custom code. Which ingestion option is the best fit?

A Use a streaming table directly against the SaaS source without a connector or credentials.

B Write a notebook that scrapes the application UI and stores screenshots in DBFS.

C Ask analysts to export CSV files manually and upload them to the workspace.

D Use Lakeflow Connect managed connectors when a supported enterprise source needs reliable, governed ingestion with minimal custom code. ✓

CORRECT: D Why
Lakeflow Connect managed connectors reduce custom ingestion code for supported sources and land data into governed tables. Manual exports or UI scraping are fragile and hard to govern.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q17

CONNECTORS & APIS

A notebook must call a REST API for payment gateway extracts and load results into Unity Catalog tables on a schedule. What security and orchestration pattern is best?

A	Use a service principal or governed credential pattern and orchestrate the REST/JDBC extraction with Lakeflow Jobs. ✓
B	Share one analyst user account across all scheduled ingestion jobs.
C	Embed a personal access token and password directly in the notebook source code.
D	Store credentials in a Delta table so every workspace user can query them.

CORRECT: A Why
Automated ingestion should use non-personal credentials and be scheduled with Jobs. Hardcoding personal secrets is insecure and brittle.

Q18

LANDING ZONE & GOVERNED TARGETS

A semi-structured ERP snapshots feed contains nested JSON and occasional malformed records. What landing pattern best supports replay, audit, and later cleaning?

A	Land raw records in a bronze Delta table first, then apply parsing and validation downstream. ✓
B	Load directly into a gold dashboard table and discard raw fields not used today.
C	Store raw records only in notebook variables for auditability.
D	Update a materialized view directly from the raw file path without preserving the source data.

CORRECT: A Why
Bronze tables preserve raw ingest for replay and audit before silver validation. Direct-to-gold discards valuable source detail and makes recovery harder.

SECTION 02 · 21% OF THE EXAM

Data Ingestion and Loading

Q19

LANDING ZONE & GOVERNED TARGETS

After ingesting CRM exports, downstream teams need governed read access without direct write access to raw storage. What target pattern should be used?

A Give every consumer cloud storage write access to the raw landing bucket.

B Create tables in the legacy Hive metastore to avoid catalog-level permissions.

C Write files to a user home folder and ask consumers to mount that path.

D Define the target as a Unity Catalog governed Delta table and grant consumers access through UC privileges. ✓

CORRECT: D Why
Unity Catalog governed Delta tables centralize access control. Granting cloud write access to raw buckets breaks least privilege and platform governance.

Q20

DATA-DRIVEN TRIGGERS

A manufacturing machine logs feed lands irregularly, and the next task should not run until new data is available. Which trigger strategy is most appropriate?

A Use a dashboard view count trigger because consumer activity proves data freshness.

B Use a cluster resize event because it indicates new source data is ready.

C Use a fixed hourly schedule even when the source arrives unpredictably and late.

D Use data-driven triggers such as file arrival or table update when downstream processing should start only after new data is available. ✓

CORRECT: D Why
File arrival or table update triggers align orchestration with data availability. Fixed schedules can run too early or waste compute.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q21

MEDALLION ARCHITECTURE & DATA QUALITY

The `bronze_iot` table contains raw strings, null customer IDs, and inconsistent timestamp formats. What is the best medallion step?

A Build a gold dashboard table directly from source files and skip silver validation.

B Overwrite the bronze table with only cleaned columns and delete the raw values.

C Read the bronze table, standardize types and null handling, then write a cleaned silver Delta table. ✓

D Store cleaned results in a temporary view because it persists after cluster termination.

CORRECT: C Why
Silver tables should contain cleaned, typed, validated records derived from bronze. Overwriting bronze removes raw lineage and replay capability.

Q22

MEDALLION ARCHITECTURE & DATA QUALITY

Before publishing `gold_sales_mart`, the team must ensure `order_amount` is nonnegative and `customer_id` is present. What should the transformation include?

A Use DESCRIBE HISTORY because it prevents invalid records from being written.

B Use VACUUM before writing because it validates primary keys.

C Apply expectations or validation rules that quarantine, fail, or flag records according to business requirements. ✓

D Use a larger cluster because data quality failures are usually memory problems.

CORRECT: C Why
Data quality checks validate records before publishing. History and VACUUM are not data-quality enforcement mechanisms.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q23

MEDALLION ARCHITECTURE & DATA QUALITY

The team is designing `gold_customer_360` for BI and analytics. Which storage format is the most appropriate default on Databricks?

A	Use Delta tables for curated silver and gold layers so downstream users get reliable ACID-backed datasets. ✓
B	Use notebook result cells because they are a persistent governed storage layer.
C	Use CSV files because BI tools require CSV rather than table formats.
D	Use unmanaged JSON folders for all gold facts because JSON is faster for aggregations.

CORRECT: A Why
Delta tables provide reliability and performance features for curated datasets. CSV/JSON folders lack the transactional table semantics expected for lakehouse BI.

Q24

JOINS & UNIONS

A transformation should keep only events records that have a matching customer record. Which DataFrame operation is appropriate?

A	Use union because it combines rows by key from two DataFrames.
B	Use a cross join because it matches rows by key automatically.
C	Use an inner join when only records with matching keys in both DataFrames should remain. ✓
D	Use a left join because it removes all unmatched rows from the left DataFrame.

CORRECT: C Why
An inner join returns only matched rows. A left join keeps all left-side rows, including unmatched rows with null right-side values.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q25

JOINS & UNIONS

A silver fact table must preserve all claims records while adding optional attributes from a small reference table. Which operation fits?

A Use cross join because it is the safest way to avoid dropping facts.

B Use an inner join because it preserves unmatched fact rows with null dimensions.

C Use union all because it enriches fact rows with dimension columns.

D Use a left join so all fact rows are preserved and matching dimension attributes are added when available. ✓

CORRECT: D Why
A left join preserves all rows from the left fact table. Inner join would drop facts without matching dimensions.

Q26

JOINS & UNIONS

A large `silver_customers` DataFrame joins to a tiny country-code lookup table. Which optimization is often appropriate?

A Use union to avoid a join and preserve all attributes automatically.

B Reduce executor memory so Spark is forced to broadcast the larger table.

C Use cross join to make Spark avoid shuffling the data.

D Use broadcast join for a small dimension table to reduce shuffle during the join. ✓

CORRECT: D Why
Broadcasting a small lookup avoids shuffling the large table. Cross joins create Cartesian products and are usually expensive and wrong.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q27

JOINS & UNIONS

Two monthly DataFrames for claims have the same column names but different column order. Which approach prevents values from being placed in the wrong columns?

A Use unionByName when combining DataFrames with the same logical columns that may be ordered differently. ✓

B Use dropDuplicates before union because it automatically reorders columns.

C Use withColumnRenamed on every column to force position-based union.

D Use cross join because it aligns columns by name.

CORRECT: A **Why**
unionByName aligns by column name. Position-based union can corrupt data if column order differs.

Q28

JOINS & UNIONS

A developer accidentally used a cross join between `silver_events` and a reference table, causing row counts to explode. What is the key lesson?

A Prefer cross joins for all lookup enrichment because they reduce row counts.

B Use cross joins to replace broadcast joins whenever a table is small.

C Use cross joins to guarantee data quality rules run before joins.

D Avoid cross joins unless a Cartesian product is explicitly required and controlled. ✓

CORRECT: D **Why**
Cross joins produce Cartesian products and should be intentional. A broadcast join may optimize a keyed join but is not the same as a cross join.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q29

TRANSFORMING DATA

A JSON record in `bronze_events` has an array of purchased items, and the team needs one row per item. Which transformation should be used?

A Use `approx_count_distinct` because it automatically normalizes nested arrays.

B Use coalesce to split each array into multiple rows.

C Use drop to remove the array because arrays cannot be processed in Spark.

D Use explode to turn each array element into its own row before aggregating or joining. ✓

CORRECT: D Why
explode converts array elements into separate rows. coalesce changes partitions or handles nulls depending on context; it does not flatten arrays.

Q30

TRANSFORMING DATA

The `silver_claims` table has duplicate records for the same business key after a replay. Which operation should be used to create a deduplicated dataset?

A Use OPTIMIZE because compaction enforces uniqueness constraints automatically.

B Use count because aggregate counts remove duplicate rows from the target.

C Use VACUUM because it deduplicates duplicate business records.

D Use dropDuplicates on the appropriate business keys, optionally ordered by timestamp when the latest record must win. ✓

CORRECT: D Why
dropDuplicates or window-based deduplication handles duplicate records by keys. OPTIMIZE and VACUUM affect file management, not logical deduplication.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q31

TRANSFORMING DATA

A gold metric needs the number of unique active users. Accuracy is important for finance, but an exploratory dashboard can tolerate approximation. Which functions fit?

A Use countDistinct for exact distinct counts or `approx_count_distinct` when an approximate result is acceptable at lower cost. ✓

B Use cross join because it makes distinct counts deterministic.

C Use union all because it removes duplicate identifiers before counting.

D Use mean because it returns a distinct count for numeric identifiers.

CORRECT: A Why
Exact and approximate distinct-count functions serve different accuracy/cost needs. mean is unrelated to counting distinct identifiers.

Q32

TRANSFORMING DATA

A CDC-like feed in `bronze_crm` contains multiple updates per `claims_id`. The silver table should keep the latest update per key. Which pattern is best?

A Use a window function to rank records by business key and timestamp, then keep the latest record. ✓

B Use repartition(1) because one partition guarantees latest-per-key semantics.

C Use union all with the previous table because it removes older versions.

D Use limit(1) on the whole DataFrame because it keeps the latest row for each key.

CORRECT: A Why
Window functions can rank rows within each business key by timestamp. limit(1) returns only one row overall, not one per key.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q33

SPARK PERFORMANCE BASICS

A join-heavy transformation on `silver_orders` shows very large shuffle tasks and long stage times. Which tuning principle is best?

A	Increase shuffle partitions when tasks are too large, or reduce them when there are too many tiny tasks, then re-measure performance. ✓
B	Set spark.sql.shuffle.partitions to 1 for all production jobs to reduce overhead.
C	Disable all shuffles because joins and aggregations never require them.
D	Change driver memory first because shuffle partition size is controlled only by the driver.

CORRECT: A Why
Shuffle partition tuning depends on task size and must be measured. Setting partitions to 1 usually creates bottlenecks.

Q34

WRITING & SERVING DATA

A BI dashboard repeatedly queries an expensive aggregate over `silver_claims`. The result should be precomputed and refreshed for users. Which gold object is appropriate?

A	Use raw bronze tables because BI performance is independent of modeling.
B	Use a materialized view when the precomputed result should be maintained for repeated BI queries. ✓
C	Use a temporary view because it is the best persistent object for BI teams.
D	Use a normal view when the result must always be physically precomputed and refreshed.

CORRECT: B Why
Materialized views store computed results for faster repeated queries. A normal view is logical and may recompute at query time.

SECTION 03 · 22% OF THE EXAM

Data Transformation and Modeling

Q35

WRITING & SERVING DATA

The team wants to expose only selected columns from `silver_orders` without materializing a new dataset. Which object best fits?

A Use a view when consumers need a logical abstraction without storing another copy of the data. ✓

B Use a managed table because every logical filter requires duplicating the data.

C Use a checkpoint because checkpoints expose SQL objects to analysts.

D Use a streaming table because all BI queries require streaming semantics.

CORRECT: A Why
Views provide a logical query abstraction. Managed tables create stored data, which may be unnecessary.

Q36

WRITING & SERVING DATA

An incremental customers feed contains both new records and updates to existing keys. What write pattern should maintain the target Delta table?

A Use merge/upsert logic when incremental changes must update existing rows and insert new rows in the Delta target. ✓

B Use append-only writes because they automatically update matching target rows.

C Use VACUUM because it applies CDC changes to Delta tables.

D Use SELECT DISTINCT because it can update changed values in place.

CORRECT: A Why
MERGE supports update/insert logic for Delta targets. Append-only loads create duplicate versions and do not update existing rows.

SECTION 04 · 16% OF THE EXAM

Working with Lakeflow Jobs

Q37

TASK DESIGN & ORCHESTRATION

A pipeline has ingestion, validation, transformation, and publication steps. Failures in validation must block publication. What orchestration design is best?

A Schedule each task independently at the same time and hope upstream data is ready.

B Put every step in one notebook and ask users to run cells manually in order.

C Use dashboard refresh order as the orchestration dependency graph.

D Model the workflow as a Lakeflow Jobs DAG with explicit task dependencies. ✓

CORRECT: D Why
A Jobs DAG represents dependencies explicitly and blocks downstream tasks after upstream failures. Independent schedules are fragile and can race.

Q38

TASK DESIGN & ORCHESTRATION

A monolithic job is hard to debug because any failure requires rerunning everything. What redesign improves maintainability?

A Use one task per SQL statement with no dependencies at all.

B Avoid Jobs and depend on a BI dashboard refresh to run transformations.

C Use separate tasks with dependencies so failed steps can be retried independently and lineage is easier to inspect. ✓

D Combine every operation into one 2,000-line notebook to reduce task count.

CORRECT: C Why
Breaking workflows into dependent tasks improves observability, retries, and isolation. A giant notebook hides step-level state and failure boundaries.

SECTION 04 · 16% OF THE EXAM

Working with Lakeflow Jobs

Q39

TASK DESIGN & ORCHESTRATION

A workflow must run an existing declarative pipeline, then execute a SQL quality check. Which task type should represent the pipeline step?

A Use an email notification task as the compute engine for a pipeline.

B Use a dashboard task because dashboards execute pipeline transformations.

C Use a pipeline task when orchestrating a Lakeflow Spark Declarative Pipeline as part of a larger workflow. ✓

D Use a SQL alert task to replace the pipeline definition.

CORRECT: C **Why**
Pipeline tasks orchestrate Lakeflow Spark Declarative Pipelines inside Jobs. Dashboard or alert tasks are not pipeline execution engines.

Editor's note Older materials call this Delta Live Tables (DLT). Expect the current Lakeflow names on the exam.

Q40

TASK DESIGN & ORCHESTRATION

A job must process the same validation logic for 40 source tables. Which orchestration pattern reduces duplication?

A Use cluster autoscaling as a loop control mechanism.

B Use cross join in Spark because it creates one job task per input value.

C Duplicate the same notebook task hundreds of times manually for each input.

D Use looping/for-each patterns when the same task logic should run over a controlled list of inputs. ✓

CORRECT: D **Why**
Looping patterns reduce repeated task definitions for similar operations. Manual duplication is error-prone and hard to maintain.

SECTION 04 · 16% OF THE EXAM

Working with Lakeflow Jobs

Q41

RETRIES, TRIGGERS & MONITORING

A notebook task occasionally fails because a source API times out for one minute. What Lakeflow Jobs feature should be used?

A Use a larger SQL warehouse because retries are not supported for tasks.

B Configure task-level retries with sensible limits and alerts for persistent failure. ✓

C Set unlimited retries with no delay so transient errors disappear from history.

D Disable run history because it makes retry behavior harder to read.

CORRECT: B Why
Retries handle transient failures while preserving observability. Unlimited immediate retries can waste compute and hide persistent defects.

Q42

RETRIES, TRIGGERS & MONITORING

A daily source sometimes arrives at 2 AM and sometimes at 5 AM. The transformation should start only when new data is present. Which trigger is best?

A Use a file arrival or table update trigger when the workflow should start after new upstream data is available. ✓

B Use a time-based trigger only, even when upstream arrival is unpredictable.

C Use a cluster termination trigger because it means upstream data is complete.

D Use a manual trigger and ask the source team to send a chat message.

CORRECT: A Why
Data-driven triggers reduce wasted runs and late failures when data arrival varies. Fixed schedules can start before data is ready.

SECTION 04 · 16% OF THE EXAM

Working with Lakeflow Jobs

Q43

RETRIES, TRIGGERS & MONITORING

A downstream publication task did not run last night. Where should an engineer look first in Lakeflow Jobs?

A	Delete the failed run to allow downstream tasks to continue.
B	Change every task to manual execution so the DAG is hidden.
C	Review run history, task duration, task status, and the DAG graph to find the blocker. ✓
D	Review only cluster size because task failures are never visible in Jobs.

CORRECT: C Why
Run history and the DAG show statuses, dependencies, and blockers. Deleting history removes evidence and does not solve the cause.

Q44

CONTROL FLOW & PARAMETERS

After a validation task checks record counts, the pipeline should publish gold data only if the check passes. Which Jobs capability fits?

A	Use conditional branching to run downstream tasks only when the validation result meets the required condition. ✓
B	Use a cluster policy because policies evaluate row-level validation results.
C	Use a fixed schedule and manually cancel downstream tasks after validation fails.
D	Use OPTIMIZE because it prevents downstream tasks from starting.

CORRECT: A Why
Conditional tasks support branching based on task outcomes or values. Cluster policies govern compute settings, not business branching.

SECTION 04 · 16% OF THE EXAM

Working with Lakeflow Jobs

Q45

CONTROL FLOW & PARAMETERS

An ingestion task calculates a `batch_id` that a downstream validation task must use. What is an appropriate Jobs pattern?

A	Use a screenshot of the previous task output as the next task input.
B	Change the job cluster name dynamically and parse it downstream.
C	Write control values into a public DBFS file with no access controls.
D	**Use task values or parameters to pass small control values between tasks.** ✓

CORRECT: D Why
Jobs parameters/task values support passing small control values. Public files or naming hacks are brittle and insecure.

Q46

CONTROL FLOW & PARAMETERS

The same pipeline should run for dev and prod with different target catalogs. Which Jobs configuration practice is best?

A	Use the cluster display name as the only source of environment configuration.
B	**Use job parameters to supply environment-specific values such as catalog, schema, or date window.** ✓
C	Hardcode production catalog names inside every notebook.
D	Ask developers to manually edit notebooks before each scheduled run.

CORRECT: B Why
Parameters make jobs reusable and environment-aware. Hardcoding prod values makes promotion risky and error-prone.

SECTION 05 · 10% OF THE EXAM

Implementing CI/CD

Q47

GIT & CODE QUALITY

A developer needs to change a transformation notebook and have it reviewed before promotion. What workflow fits Databricks Git integration?

A Edit the production notebook directly and rely on workspace revision history as the pull request.

B Use Databricks Git Folders to create a branch, commit changes, push, and open a pull request in the Git provider. ✓

C Copy notebooks between user folders by hand after each change.

D Store code only in job run output logs because logs are version controlled.

CORRECT: B Why
Git Folders support branch-based development, commits, pushes, and PR workflows. Workspace revision history is not a substitute for Git review and promotion.

Q48

GIT & CODE QUALITY

Several pipelines need the same data quality functions. What development practice reduces duplication?

A Keep source code in Git and package reusable Python logic as libraries when multiple jobs need the same transformations. ✓

B Require analysts to copy functions from dashboard descriptions.

C Store function source code in table comments and eval it at runtime.

D Paste the same functions into every production notebook.

CORRECT: A Why
Shared libraries and Git versioning improve reuse and review. Copy-paste logic causes inconsistent bug fixes.

SECTION 05 · 10% OF THE EXAM

Implementing CI/CD

Q49

GIT & CODE QUALITY

A transformation change affects revenue calculations. Which practice best reduces deployment risk?

A Delete the feature branch without review and redeploy old code.

B Use a pull request and automated tests before merging changes to the deployment branch. ✓

C Treat successful cluster startup as proof the transformation is correct.

D Merge directly to production after a notebook runs once on a sample.

CORRECT: B **Why**
PR review and tests catch logic issues before promotion. A single sample run or cluster startup check is not enough.

Q50

BUNDLES & DEPLOYMENT

A team wants the same Lakeflow Jobs and pipeline resources deployed consistently across dev, test, and prod. What should they use?

A Use Declarative Automation Bundles to define jobs, pipelines, permissions, and variables for repeatable deployment. ✓

B Use Unity Catalog table comments as the deployment manifest.

C Export notebooks as HTML and attach them to a ticket for deployment.

D Create resources manually in each workspace and document screenshots for auditors.

CORRECT: A **Why**
Declarative Automation Bundles package resource definitions and support target-specific deployment. Manual clicks are difficult to reproduce and audit.

Editor's note Declarative Automation Bundles is the current name of Databricks Asset Bundles (DABs). The rename is non-breaking.

SECTION 05 · 10% OF THE EXAM

Implementing CI/CD

Q51

BUNDLES & DEPLOYMENT

The same codebase needs different catalog names and compute settings in dev and prod. Which CI/CD pattern is best?

A Hardcode prod object names and tell developers not to run dev jobs.

B Maintain three unrelated codebases, one for each environment.

C Use notebook widgets only and never define deployment configuration.

D Use bundle targets, variables, and overrides for environment-specific catalog, schema, warehouse, and job settings. ✓

CORRECT: D Why
Bundle variables/targets separate configuration from code. Separate codebases drift and make promotion unreliable.

Q52

BUNDLES & DEPLOYMENT

A CI pipeline should fail fast when a bundle definition is invalid. Which CLI step is most relevant?

A Skip validation because deployment always fixes invalid YAML automatically.

B Only test by opening the workspace UI after production deployment.

C Run VACUUM on the repository before deploying.

D Run bundle validate before deploy to catch configuration errors early in the pipeline. ✓

CORRECT: D Why
bundle validate checks the bundle configuration before deployment. UI-only testing after production deployment is late and risky.

SECTION 05 · 10% OF THE EXAM

Implementing CI/CD

Q53

BUNDLES & DEPLOYMENT

The team wants dev, test, and prod deployments to be traceable and consistent. Which promotion model is best?

A Promote the same artifact or bundle definition through environments with controlled configuration changes. ✓

B Export prod notebooks back to dev so dev follows production drift.

C Use separate unmanaged workspaces with no shared source control.

D Recreate the job manually in prod with a slightly different notebook path.

CORRECT: A Why
Promoting the same versioned definition improves traceability. Manual recreation causes drift and makes rollback harder.

Q54

BUNDLES & DEPLOYMENT

A GitHub Actions workflow needs to deploy Databricks resources after merge. Which tool is intended for this automation?

A Use Databricks CLI in the CI runner to validate, deploy, and manage bundle resources. ✓

B Use a BI dashboard export as the bundle deployment tool.

C Use the Spark UI as a deployment client because it manages bundles.

D Use Delta OPTIMIZE to push code to workspaces.

CORRECT: A Why
The Databricks CLI supports bundle validation and deployment from CI systems. Spark UI and dashboard exports are not deployment clients.

SECTION 05 · 10% OF THE EXAM

Implementing CI/CD

Q55

IDENTITY & SECRETS

Production deployments must be automated and auditable. Which identity pattern is most appropriate?

A Give all workspace users owner permissions to simplify deployment.

B Ask each engineer to deploy manually when they finish coding.

C Use one developer laptop as the production deployment server with a personal token.

D Deploy through CI/CD using a service principal with appropriate permissions instead of a developer personal identity. ✓

CORRECT: D Why
Service principals support controlled, non-personal automation. Personal tokens and manual deployment reduce auditability and continuity.

Q56

IDENTITY & SECRETS

A bundle deploys a job that reads from an external API. How should sensitive credentials be handled?

A Place secrets in notebook markdown cells so they are hidden from job runs.

B Avoid storing secrets in source code; reference secret scopes or governed credentials through configuration. ✓

C Commit API keys into the repository because private repos are always secure enough.

D Put passwords in Delta table comments because comments are not queryable.

CORRECT: B Why
Secrets should not be committed to source. Use secret management or governed credentials. Private repos do not remove insider or leakage risk.

Troubleshooting, Monitoring, and Optimization

Q57

SPARK UI & PERFORMANCE ISSUES

A Spark stage has 300 tasks: most finish in 20 seconds, but one task runs for 15 minutes and reads far more shuffle data than the median. What is the likely issue and remedy?

A	The cluster has too few notebooks attached; attach more notebooks to increase parallelism.
B	The problem is definitely missing VACUUM; vacuuming the target removes skewed shuffle reads.
C	The table needs more columns because wide rows always reduce skew.
D	**Data skew is likely; enable/confirm AQE skew handling or apply salting/repartitioning for the skewed key. ✓**

CORRECT: D Why
One extreme task with much higher shuffle read is a classic skew signal. Adding notebooks or vacuuming files does not fix skewed keys.

Q58

SPARK UI & PERFORMANCE ISSUES

A transformation spends most time in stages with high shuffle read/write and many spilled records. What should the engineer investigate?

A	**Shuffle is expensive; reduce unnecessary wide transformations, tune partitions, or optimize the join/aggregation strategy. ✓**
B	The SQL warehouse cache must be disabled because cache causes all shuffles.
C	The job needs a dashboard task to eliminate shuffle.
D	Driver logs are large, so delete them before running the job.

CORRECT: A Why
High shuffle and spill point to wide transformations, partition sizing, memory pressure, or join strategy. Dashboard tasks do not remove Spark shuffle.

Troubleshooting, Monitoring, and Optimization

Q59

SPARK UI & PERFORMANCE ISSUES

A job fails with executor out-of-memory while processing a large aggregation. Which response is most appropriate?

A Increase appropriate executor memory or reduce per-task data size; investigate spills and partitioning. ✓

B Increase only the driver memory because executor OOM can only be fixed on the driver.

C Disable the transaction log because it uses executor heap.

D Set shuffle partitions to 1 so each task has complete context.

CORRECT: A Why
Executor OOM requires addressing executor memory, task size, shuffles, and partitioning. Driver memory alone does not fix executor heap pressure.

Q60

SPARK UI & PERFORMANCE ISSUES

A job completes successfully but slowly. Which tool and metric set best supports stage-level performance analysis?

A Use Spark UI stage metrics such as task duration, shuffle read/write, spill, and input size to identify bottlenecks. ✓

B Use Unity Catalog privileges to infer shuffle read size.

C Use only the workspace home page because it shows detailed Spark stage metrics.

D Use Git commit count as the primary performance metric.

CORRECT: A Why
Spark UI stage metrics expose task duration, shuffle, spill, and input patterns. Access privileges and Git metadata do not diagnose stage bottlenecks.

SECTION 06 · 10% OF THE EXAM

Troubleshooting, Monitoring, and Optimization

Q61

CLUSTER & DEPENDENCY ISSUES

A job never starts because the cluster fails during initialization. What should be checked first?

- **A** Check cluster event logs, init scripts, libraries, policies, and cloud capacity or permissions related to startup. ✓
- **B** Check only the output Delta table because table contents determine cluster startup.
- **C** Delete the job run history to force a fresh startup.
- **D** Run OPTIMIZE on all tables before starting the cluster.

CORRECT: A Why
Startup failures often involve environment, libraries, init scripts, policies, or cloud capacity/permissions. Table optimization does not resolve cluster initialization.

Q62

CLUSTER & DEPENDENCY ISSUES

A notebook worked yesterday, but after installing a new package it fails during imports with version conflicts. What is the best diagnosis?

- **A** Liquid clustering must be disabled because it controls notebook imports.
- **B** A row-level security policy is causing Python imports to fail.
- **C** A dependency conflict is likely; align library versions with the Databricks Runtime and install only required compatible libraries. ✓
- **D** The dashboard refresh interval is too frequent for pip.

CORRECT: C Why
Library version conflicts commonly break imports. Governance policies do not normally cause Python dependency import conflicts.

Troubleshooting, Monitoring, and Optimization

Q63

MONITORING JOBS & PIPELINES

A nightly pipeline is getting slower over the past two weeks. Where should the engineer begin?

- **A** Use VACUUM logs as the authoritative job performance trend source.
- **B** Use notebook cell output screenshots because they are the Jobs monitoring system.
- **C** Use only DESCRIBE DETAIL on the final table because it includes every job retry reason.
- **D** Use Lakeflow Jobs run history to compare durations, statuses, retries, and failures against historical baselines. ✓

CORRECT: D Why
Jobs run history provides durations, statuses, and trends. Table metadata alone is not a complete workflow performance history.

Q64

MONITORING JOBS & PIPELINES

A gold publication task did not execute, but no error appears in that task's logs. What should be inspected?

- **A** Use the DAG graph to identify failed upstream tasks or dependencies that blocked downstream execution. ✓
- **B** Delete task dependencies so every task runs regardless of data readiness.
- **C** Restart every cluster in the workspace because blocked DAG tasks always mean workspace outage.
- **D** Manually run only the final task and ignore upstream failures.

CORRECT: A Why
A task can be skipped or blocked because an upstream dependency failed. The DAG makes dependency blockers visible.

Troubleshooting, Monitoring, and Optimization

Q65

MONITORING JOBS & PIPELINES

A pipeline duration doubled after a new upstream source was added. What diagnostic approach is most useful?

A Delete historical run metrics because old baselines are irrelevant.

B Immediately rewrite the entire pipeline in another language.

C Compare current run metrics to prior successful runs to isolate whether input volume, skew, shuffle, or environment changed. ✓

D Assume every slowdown is caused by missing permissions.

CORRECT: C Why
Historical comparison helps isolate changes in data volume, skew, shuffles, failures, or environment. Rewriting first is premature.

Q66

DELTA TABLE OPTIMIZATION

A large Delta table serves queries with evolving filter patterns, and static partitioning is becoming hard to maintain. Which feature is relevant?

A Liquid clustering replaces the need for any access controls on sensitive tables.

B Liquid clustering can help maintain clustering for query performance without manually managing static partitions for changing workloads. ✓

C Liquid clustering is used to deduplicate rows during ingestion.

D Liquid clustering is a job scheduler trigger type.

CORRECT: B Why
Liquid clustering is a table layout optimization feature. It does not handle access control, deduplication, or scheduling.

SECTION 06 · 10% OF THE EXAM

Troubleshooting, Monitoring, and Optimization

Q67

DELTA TABLE OPTIMIZATION

A platform team wants Databricks to automatically apply supported table optimization maintenance where available. What feature matches this goal?

A Predictive optimization grants users SELECT permissions based on query frequency.

B Predictive optimization guarantees no job can fail from bad code.

C Predictive optimization converts every external table into an in-memory view.

D Predictive optimization can automate supported maintenance operations to improve table performance with less manual tuning. ✓

CORRECT: D Why
Predictive optimization automates eligible maintenance/optimization tasks. It is not an access-control or code-correctness feature.

Q68

DELTA TABLE OPTIMIZATION

A table receives many small micro-batches and BI queries are slower than expected. What performance issue should be considered?

A Create more tiny files because each file becomes a faster Spark task.

B Check whether too many tiny files are hurting query planning and scan efficiency, then compact/optimize where appropriate. ✓

C Turn off Delta transaction logs because logs slow down only tiny files.

D Use cross join to combine files before reading them.

CORRECT: B Why
Many tiny files can increase overhead; compaction/optimization can improve layout. More tiny files usually worsen, not improve, scan efficiency.

SECTION 07 · 15% OF THE EXAM

Governance and Security

Q69

MANAGED VS EXTERNAL TABLES

A team wants Databricks to manage table metadata and storage lifecycle for a curated dataset. Which table type fits?

A Use a dashboard because dashboards are the storage layer for governed tables.

B Use a managed table when Databricks should manage both metadata and underlying data storage lifecycle. ✓

C Use a temporary view because it manages durable storage.

D Use an external table because Databricks always deletes external data when the table is dropped.

CORRECT: B Why
Managed tables are managed by Databricks/Unity Catalog for metadata and storage lifecycle. External tables point to externally managed storage.

Q70

MANAGED VS EXTERNAL TABLES

A data lake team owns an existing cloud path and wants Databricks to query it without taking over storage lifecycle. What should be created?

A Use a managed table because dropping it never affects underlying data.

B Use a temp view because it preserves external storage governance permanently.

C Use a job cluster because clusters define external table metadata.

D Use an external table when data remains in an external cloud storage location managed outside the table lifecycle. ✓

CORRECT: D Why
External tables reference data in external locations. Managed tables are not the right choice when external storage lifecycle must remain separate.

SECTION 07 · 15% OF THE EXAM

Governance and Security

Q71

PRIVILEGES & IDENTITIES

A group of analysts needs read access to one view containing curated sales data. What access-control principle should be applied?

A Use personal access tokens as the main table authorization mechanism.

B Grant the minimum required privilege at the narrowest appropriate scope to a group, not individual users whenever possible. ✓

C Grant ALL PRIVILEGES at the metastore to every analyst for simplicity.

D Grant permissions only to clusters because table permissions are inherited from clusters.

CORRECT: B Why
Least privilege and group-based grants are easier to manage and audit. Broad metastore-level grants create unnecessary exposure.

Q72

PRIVILEGES & IDENTITIES

An engineer must allow one group to SELECT a table and prevent another group from accessing it. Which SQL concepts are relevant?

A Use GRANT to allow access and REVOKE or DENY to remove or explicitly block access according to the privilege model. ✓

B Use DROP COLUMN to revoke user permissions.

C Use VACUUM to deny access to old files only.

D Use OPTIMIZE to remove users from a table.

CORRECT: A Why
Unity Catalog uses privileges such as GRANT, REVOKE, and DENY. Table maintenance commands do not manage user authorization.

Editor's note Unity Catalog documentation describes GRANT and REVOKE for managing privileges. DENY belongs to the legacy Hive metastore table ACL model, so focus on GRANT and REVOKE for the exam.

SECTION 07 · 15% OF THE EXAM

Governance and Security

Q73

PRIVILEGES & IDENTITIES

A production ingestion job writes to a bronze schema and reads an external location. What identity pattern is best?

A Use service principals for automated jobs and grant them only the privileges required for the workflow. ✓

B Run production jobs as the workspace admin to avoid permission errors.

C Use a shared human account for every scheduled pipeline.

D Grant ownership of all catalogs to the job cluster.

CORRECT: A Why
Service principals are appropriate for automation with least privilege. Admin or shared human accounts reduce accountability and increase risk.

Q74

PRIVILEGES & IDENTITIES

A data steward is deciding whether to grant permissions at catalog, schema, or table level. What concept matters most?

A Use file names to encode user privileges.

B Use Unity Catalog object hierarchy and inheritance carefully, granting at catalog/schema/table levels based on scope. ✓

C Grant only on clusters because clusters own all schemas.

D Grant only on notebooks because notebook permissions automatically grant table SELECT.

CORRECT: B Why
Unity Catalog privileges can be scoped through the object hierarchy. Notebook or cluster permissions do not replace data-object privileges.

SECTION 07 · 15% OF THE EXAM

Governance and Security

Q75

FINE-GRAINED ACCESS CONTROL

A table contains regional sales records, and analysts should see only their assigned region. What governance feature fits?

A Use OPTIMIZE ZORDER because clustering hides rows from unauthorized groups.

B Use table comments because comments prevent users from querying restricted rows.

C Use a larger warehouse so security filters are evaluated outside SQL.

D Use row-level security policies so users see only rows allowed for their group or attributes. ✓

CORRECT: D Why
Row-level security restricts rows based on identity or attributes. Physical layout optimization does not enforce access restrictions.

Q76

FINE-GRAINED ACCESS CONTROL

A customer table includes email and tax identifiers. Most analysts can query the table but should not see raw sensitive values. What should be used?

A Use column masking to hide or transform sensitive column values based on user identity or group. ✓

B Use a dashboard filter because filters secure the underlying table for all access paths.

C Use table partitioning because partitions mask sensitive values automatically.

D Rename the sensitive column to a vague name and keep granting SELECT.

CORRECT: A Why
Column masks protect sensitive values while allowing authorized use of the table. Renaming columns or dashboard filters do not secure direct table access.

SECTION 07 · 15% OF THE EXAM

Governance and Security

Q77

FINE-GRAINED ACCESS CONTROL

A privacy team wants centralized policies for many tagged PII columns across tables. Which Unity Catalog capability is most relevant?

A Create a separate hardcoded view for every user and never update it.

B Use notebook widgets because widgets centrally govern all SQL access.

C Use cluster tags as the only way to mask columns.

D Use Unity Catalog ABAC policies to centrally apply row filters or column masks based on governed attributes/tags. ✓

CORRECT: D Why
ABAC policies centralize controls based on attributes/tags. Per-user hardcoded views do not scale and are easy to drift.

Q78

GOVERNED DATA SHARING

Downstream teams want access to curated data, but the platform team must preserve auditing and centralized permissions. What pattern is best?

A Use screenshots of dashboards as the governed data sharing format.

B Publish CSV copies to personal folders because personal folders are centrally governed tables.

C Give every consumer direct storage credentials to raw buckets and skip table privileges.

D Register data in Unity Catalog and avoid bypassing governance with direct unmanaged cloud paths for consumers. ✓

CORRECT: D Why
Unity Catalog governed tables/views preserve centralized access and audit. Direct raw storage access bypasses platform-level data governance.

PART B · 28 EXTRA QUESTIONS

Hands-on SQL & PySpark

Real exam questions often show a few lines of code. These questions were **written by the editor** because the source bank has none. Each one is tagged with the exam section it belongs to (S1 to S7).

Q79

S1 · SQL · TIME TRAVEL

Which query returns the table `sales` as it was at version 5 of its Delta log?

A `SELECT * FROM sales WHERE _version = 5`

B `SELECT * FROM sales VERSION AS OF 5` ✓

C `SELECT * FROM HISTORY(sales, 5)`

D `DESCRIBE HISTORY sales LIMIT 5`

CORRECT: B Why
VERSION AS OF (or TIMESTAMP AS OF) queries a previous snapshot. DESCRIBE HISTORY only lists the operations; it does not return table rows.

Q80

S1 · SQL · RESTORE

A bad job overwrote `sales`. Which statement brings the table back to version 5?

A `RESTORE TABLE sales TO VERSION AS OF 5` ✓

B `ROLLBACK TABLE sales TO 5`

C `ALTER TABLE sales RESTORE VERSION 5`

D `UNDO TABLE sales VERSION 5`

CORRECT: A Why
RESTORE is the Delta Lake command that reverts a table to an earlier version. The restore itself is recorded as a new commit in the history.

Hands-on SQL & PySpark

Q81

S2 · PYSPARK · AUTO LOADER

The stream below infers the schema of incoming JSON files. Which option is `required` so Auto Loader can store and evolve the inferred schema?

```
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .load("/Volumes/main/raw/orders"))
```

A `checkpointLocation on the reader`

B `cloudFiles.maxFilesPerTrigger`

C `mergeSchema = true on the reader`

D `cloudFiles.schemaLocation` ✓

CORRECT: D **Why**
Auto Loader needs cloudFiles.schemaLocation to persist the inferred schema when no explicit schema is given. checkpointLocation belongs to writeStream.

Hands-on SQL & PySpark

Q82

S2 · SQL · COPY INTO

This command runs twice in a row and no new files arrive between the runs. What does the second run do?

```
COPY INTO main.bronze.orders
FROM '/Volumes/main/raw/orders'
FILEFORMAT = JSON
```

A	It reloads every file and duplicates the rows
B	It fails because the table already contains data
C	It loads nothing, because files that were already loaded are skipped ✓
D	It truncates the table and loads all files again

CORRECT: C Why

COPY INTO is idempotent: it tracks loaded files and skips them on later runs, so re-running it is safe.

Hands-on SQL & PySpark

Q83

S2 · PYSPARK · TRIGGERS

What does `trigger(availableNow=True)` do in the write below?

```
query = (df.writeStream
  .option("checkpointLocation", "/Volumes/main/chk/orders")
  .trigger(availableNow=True)
  .toTable("main.bronze.orders"))
```

A	Runs one micro-batch every second forever
B	**Processes all data available now, in batches, then stops the query** ✓
C	Processes only the newest file and stops
D	Keeps the cluster running until a new file arrives

CORRECT: B Why
availableNow processes everything that is currently available (possibly in several micro-batches) and then stops. It is the usual choice for incremental, scheduled ingestion.

Hands-on SQL & PySpark

Q84

S2 · SQL · READ_FILES

What does this query do?

```
SELECT *
FROM read_files(
  '/Volumes/main/raw/orders',
  format => 'csv',
  header => true)
```

A	Reads the CSV files directly and returns them as a table, inferring the schema ✓
B	Creates a managed table named `read_files`
C	Copies the files into a Delta table
D	Registers the volume as an external location

CORRECT: A Why

`read_files` is a table-valued function that reads files from storage in a query. It does not create or write anything by itself.

Hands-on SQL & PySpark

Q85

S3 · SQL · MERGE

A row in `updates` has an id that does not exist in the target. What happens to that row?

```
MERGE INTO main.silver.customers t
USING updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

A It is ignored

B The statement fails with a key error

C It updates the first row of the target

D It is inserted into the target table ✓

CORRECT: D Why
WHEN NOT MATCHED THEN INSERT adds source rows that have no match in the target. Matched rows are updated.

Q86

S3 · PYSPARK · DEDUPLICATION

Which line keeps exactly one row per `order_id`, even when other columns differ between duplicates?

A `df.distinct()`

B `df.groupBy("order_id")`

C `df.dropDuplicates(["order_id"])` ✓

D `df.select("order_id").dropna()`

CORRECT: C Why
dropDuplicates with a column list compares only those columns. distinct() compares whole rows, so rows that differ in any column all stay.

Hands-on SQL & PySpark

Q87

S3 · PYSPARK · WINDOW

What does the result contain?

```
from pyspark.sql import Window, functions as F

w = Window.partitionBy("customer_id").orderBy(F.col("updated_at").desc())
latest = (df.withColumn("rn", F.row_number().over(w))
            .filter("rn = 1"))
```

A	The oldest row for each customer
B	The most recent row for each customer ✓
C	Only one row in total
D	Every row, with a new column named rn

CORRECT: B Why

`row_number()` ranks rows inside each `customer_id`, newest first. Filtering rn = 1 keeps the latest one. This is a common pattern for deduplicating CDC data.

Hands-on SQL & PySpark

Q88

S3 · PYSPARK · JOINS

Some orders have no matching customer. What happens to those orders in this join?

```
result = orders.join(customers, "customer_id", "left")
```

- **A** They are kept, with nulls in the customer columns ✓
- **B** They are dropped
- **C** The join fails
- **D** They are kept, with the customer columns filled from the first customer

CORRECT: A Why
A left join keeps every row of the left side. Columns from the right side are null when there is no match.

Q89

S3 · SQL · ARRAYS

The column `items` is an array. What does this query return?

```
SELECT id, explode(items) AS item
FROM orders
```

- **A** One row per order, with items as a string
- **B** One column that holds the array length
- **C** Only the first element of each array
- **D** One row per array element, repeating the id ✓

CORRECT: D Why
explode turns each array element into its own row. Rows with an empty or null array disappear (use `explode_outer` to keep them).

Hands-on SQL & PySpark

Q90

S3 · SQL · CTAS

The table already exists. What does this statement do?

```
CREATE OR REPLACE TABLE main.gold.daily_sales AS
SELECT order_date, SUM(amount) AS total
FROM main.silver.orders
GROUP BY order_date
```

A Fails because the table exists

B Appends the new rows to the existing data

C Replaces the table atomically with the new result and keeps its Delta history ✓

D Drops the table first, so earlier versions are lost

CORRECT: C Why
CREATE OR REPLACE TABLE swaps the table content in one atomic operation. For Delta tables the earlier versions stay in the history.

Hands-on SQL & PySpark

Q91 S3 · LAKEFLOW · STREAMING TABLE

What does the STREAM keyword do in this definition?

```
CREATE OR REFRESH STREAMING TABLE bronze_orders AS
SELECT * FROM STREAM read_files(
  '/Volumes/main/raw/orders', format => 'json')
```

A	Forces a full reload of all files on each update
B	Reads the source incrementally and processes only new files on each update ✓
C	Runs the query on a continuous cluster that never stops
D	Converts the output into a temporary view

CORRECT: B Why
STREAM makes the read incremental. A streaming table processes only the new data on each pipeline update.

Hands-on SQL & PySpark

Q92

S3 · LAKEFLOW · EXPECTATIONS

A row arrives with `order_id` = NULL. What happens?

```
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL)
    ON VIOLATION DROP ROW
) AS SELECT * FROM STREAM bronze_orders
```

A	The row is dropped and the violation is counted in the pipeline quality metrics ✓
B	The whole pipeline update fails
C	The row is kept and a warning is logged only
D	The row is moved to a quarantine table automatically

CORRECT: A Why
ON VIOLATION DROP ROW removes the bad row and records it in the metrics. FAIL UPDATE would stop the update, and with no ON VIOLATION clause the row is kept and only counted.

Q93

S4 · PYTHON · TASK VALUES

Task `ingest` sets a value for downstream tasks. Which call reads it in a later notebook task?

```
dbutils.jobs.taskValues.set(key="row_count", value=100)
```

A	`dbutils.jobs.get("ingest", "row_count")`
B	`dbutils.widgets.get("ingest.row_count")`
C	`spark.conf.get("ingest.row_count")`
D	`dbutils.jobs.taskValues.get(taskKey="ingest", key="row_count")` ✓

CORRECT: D Why
Task values pass small results between tasks of the same job run. The reader names the upstream taskKey and the key.

Hands-on SQL & PySpark

Q94

S4 · PYTHON · PARAMETERS

A job passes the parameter `run_date` to a notebook task. How does the notebook read it?

A `dbutils.jobs.get("run_date")`

B `spark.read.parameter("run_date")`

C `dbutils.widgets.get("run_date")` ✓

D `os.environ["run_date"]`

CORRECT: C **Why**

Job and task parameters arrive in a notebook as widgets, so dbutils.widgets.get reads them.

Hands-on SQL & PySpark

Q95

S5 · CLI · BUNDLES

Which command deploys the bundle to the `prod` target?

```
bundle:
  name: etl_project
targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
```

A `databricks deploy --env prod`

B `databricks bundle deploy -t prod` ✓

C `databricks bundle push prod`

D `databricks bundle run prod`

CORRECT: B Why
The -t (target) flag selects the target defined in databricks.yml. bundle run starts a resource after it is deployed.

PART B · 28 EXTRA QUESTIONS

Hands-on SQL & PySpark

Q96

S5 · CLI · BUNDLES

The bundle already contains a job named `nightly_etl`. Which command starts that job in the dev target?

A `databricks bundle run -t dev nightly_etl` ✓

B `databricks jobs deploy nightly_etl --dev`

C `databricks bundle start dev nightly_etl`

D `databricks bundle validate nightly_etl`

CORRECT: A Why
bundle run starts a deployed job or pipeline. bundle validate only checks the configuration and does not run anything.

Q97

S6 · SQL · OPTIMIZE

What is the main purpose of this statement?

```
OPTIMIZE main.silver.orders
ZORDER BY (customer_id)
```

A Remove files older than the retention period

B Rebuild the table schema

C Create a secondary index on `customer_id`

D Compact small files and co-locate data by `customer_id` for faster filters ✓

CORRECT: D Why
OPTIMIZE compacts small files and ZORDER clusters related values together so queries can skip more files. For new tables, liquid clustering is the newer recommended approach.

Hands-on SQL & PySpark

Q98

S6 · SQL · VACUUM

What is the effect of this statement?

```
VACUUM main.silver.orders RETAIN 168 HOURS
```

A Compacts small files into larger ones

B Deletes all old versions immediately

C Deletes unreferenced data files older than 7 days, which limits time travel to that window ✓

D Restores the table to 7 days ago

CORRECT: C Why
VACUUM removes data files that are no longer referenced and older than the retention period. After that, versions that needed those files cannot be read.

Q99

S6 · SQL · HISTORY

Which command lists the versions, timestamps and operations of a Delta table?

A `SHOW VERSIONS main.silver.orders`

B `DESCRIBE HISTORY main.silver.orders` ✓

C `DESCRIBE DETAIL main.silver.orders`

D `SHOW TBLPROPERTIES main.silver.orders`

CORRECT: B Why
DESCRIBE HISTORY returns the transaction log summary. DESCRIBE DETAIL returns table metadata such as size and number of files.

Hands-on SQL & PySpark

Q100

S6 · PYSPARK · PARTITIONS

You want to write fewer output files without a full shuffle. Which call is the better fit?

```
df._____(10).write.format("delta").save(path)
```

A coalesce ✓

B repartition

C distinct

D cache

CORRECT: A **Why**
coalesce reduces the partition count by merging existing partitions, without a full shuffle. repartition performs a full shuffle.

Q101

S6 · PYSPARK · BROADCAST JOIN

Why use `broadcast` here?

```
from pyspark.sql import functions as F

result = orders.join(F.broadcast(dim_small), "product_id")
```

A It caches the large table in memory

B It splits the small table into many partitions

C It makes the join run on the driver

D It sends the small table to every executor so the large table is not shuffled ✓

CORRECT: D **Why**
A broadcast join copies the small side to all executors. This avoids shuffling the large table, which usually makes the join faster.

Hands-on SQL & PySpark

Q102

S7 · SQL · PRIVILEGES

The group `analysts` got this grant but still cannot query the table. What is most likely missing?

```
GRANT SELECT ON TABLE main.gold.sales TO `analysts`
```

- **A** MODIFY on the table
- **B** OWNERSHIP of the catalog
- **C** USE CATALOG on main and USE SCHEMA on main.gold ✓
- **D** CREATE TABLE on the schema

CORRECT: C Why
In Unity Catalog a user needs USE CATALOG and USE SCHEMA on the parents, in addition to SELECT on the table.

Q103

S7 · SQL · COLUMN MASKS

Which statement applies the masking function `mask_salary` to the column `salary`?

- **A** `GRANT MASK mask_salary ON main.hr.employees.salary`
- **B** `ALTER TABLE main.hr.employees ALTER COLUMN salary SET MASK mask_salary` ✓
- **C** `ALTER TABLE main.hr.employees ADD MASK mask_salary`
- **D** `CREATE MASK mask_salary ON main.hr.employees(salary)`

CORRECT: B Why
Column masks are attached with ALTER COLUMN ... SET MASK. The masking function decides what each user sees.

Hands-on SQL & PySpark

Q104

S7 · SQL · EXTERNAL TABLES

What happens to the data files when this runs on an external table?

```
DROP TABLE main.raw.events
```

- **A** The metadata is removed and the files stay in cloud storage ✓
- **B** The files are deleted from cloud storage
- **C** The files are moved to the trash for 7 days
- **D** The statement fails for external tables

CORRECT: A Why
Dropping an external table removes only the metadata. Dropping a managed table also deletes the data.

Q105

S7 · SQL · DELTA SHARING

A share was created and a table was added. What must be done next so the partner can read it?

```
CREATE SHARE partner_share;
ALTER SHARE partner_share ADD TABLE main.gold.sales;
```

- **A** Email the partner a copy of the table files
- **B** Make the table an external table
- **C** Give the partner USE CATALOG on main
- **D** Create a recipient and grant it SELECT on the share ✓

CORRECT: D Why
A share is made available by creating a recipient and granting it access to the share with GRANT SELECT ON SHARE.

Hands-on SQL & PySpark

Q106

S7 · PYTHON · SECRETS

What does a notebook print for this code?

```
password = dbutils.secrets.get(scope="kv", key="db-password")
print(password)
```

A The secret value in plain text

B An empty string

C [REDACTED] ✓

D A hash of the secret

CORRECT: C **Why**
Databricks redacts secret values in notebook output. The value is still usable in code, for example to open a connection.

QUICK CHECK

Answer key

Q1 **D**	Q2 **D**	Q3 **B**	Q4 **A**	Q5 **C**	Q6 **A**
Q7 **D**	Q8 **C**	Q9 **B**	Q10 **B**	Q11 **D**	Q12 **C**
Q13 **C**	Q14 **D**	Q15 **A**	Q16 **D**	Q17 **A**	Q18 **A**
Q19 **D**	Q20 **D**	Q21 **C**	Q22 **C**	Q23 **A**	Q24 **C**
Q25 **D**	Q26 **D**	Q27 **A**	Q28 **D**	Q29 **D**	Q30 **D**
Q31 **A**	Q32 **A**	Q33 **A**	Q34 **B**	Q35 **A**	Q36 **A**
Q37 **D**	Q38 **C**	Q39 **C**	Q40 **D**	Q41 **B**	Q42 **A**
Q43 **C**	Q44 **A**	Q45 **D**	Q46 **B**	Q47 **B**	Q48 **A**
Q49 **B**	Q50 **A**	Q51 **D**	Q52 **D**	Q53 **A**	Q54 **A**
Q55 **D**	Q56 **B**	Q57 **D**	Q58 **A**	Q59 **A**	Q60 **A**
Q61 **A**	Q62 **C**	Q63 **D**	Q64 **A**	Q65 **C**	Q66 **B**
Q67 **D**	Q68 **B**	Q69 **B**	Q70 **D**	Q71 **B**	Q72 **A**
Q73 **A**	Q74 **B**	Q75 **D**	Q76 **A**	Q77 **D**	Q78 **D**
Q79 **B**	Q80 **A**	Q81 **D**	Q82 **C**	Q83 **B**	Q84 **A**

QUICK CHECK

Answer key

Q85 **D**	Q86 **C**	Q87 **B**	Q88 **A**	Q89 **D**	Q90 **C**
Q91 **B**	Q92 **A**	Q93 **D**	Q94 **C**	Q95 **B**	Q96 **A**
Q97 **D**	Q98 **C**	Q99 **B**	Q100 **A**	Q101 **D**	Q102 **C**
Q103 **B**	Q104 **A**	Q105 **D**	Q106 **C**		

Cell colors match the section colors in the table of contents.

DEDUPLICATION REPORT

1,030 questions, 78 unique concepts

SECTION	SOURCE Q&A	UNIQUE KEPT	REPEATS REMOVED
Databricks Intelligence Platform	120	8	112
Data Ingestion and Loading	205	12	193
Data Transformation and Modeling	245	16	229
Working with Lakeflow Jobs	120	10	110
Implementing CI/CD	100	10	90
Troubleshooting, Monitoring, and Optimization	135	12	123
Governance and Security	105	10	95
Total	1030	78	952

Method. Questions were grouped by identical correct answer and explanation. Inside each group the stems differed only by industry, data source, cloud provider or a "Scenario variant" tag, so one clean version per group was kept.

Related but kept. A few questions touch the same idea under different exam objectives, so they stay:

- Q1, Q3 and Q23: Delta tables governed by Unity Catalog (platform, lakehouse, medallion).
- Q20 and Q42: data-driven triggers (ingestion vs. Lakeflow Jobs).
- Q43 and Q64: using the run history and the DAG to find what blocked a task.
- Q55 and Q73: service principals for automation (deployment vs. ingestion).
- Q19 and Q78: governed access to curated data through Unity Catalog.

Good luck on the exam!

Practice, review the explanations, and get hands-on in a real workspace.

Stanly Fernández

Business Intelligence & Data Analytics

Power BI

SQL

Python

Databricks

Personal data project on GitHub

github.com/Stanly06/rd-economic-financial-intelligence

Practice questions are adapted from cert-pass.com (see the credits page).
Databricks and related names are trademarks of Databricks, Inc.